

1. INTELLIGENT AGENTS IN A SMART HOME AUTOMATION SYSTEM

Introduction

An **intelligent agent** is a system that **perceives its environment through sensors and takes actions through actuators** to achieve a desired result.

In a smart home, sensors may detect **motion, temperature, light, door status, and time**, while actuators control **lights, fans, air conditioners, alarms, and locks**.

The four important types of agents are:

1. Simple Reflex Agent
2. Model-Based Agent
3. Goal-Based Agent
4. Utility-Based Agent

1. Simple Reflex Agent

A **simple reflex agent** makes decisions based only on the **current percept**. It follows predefined **IF–THEN rules**.

Working in a smart home

Examples:

- **IF** motion is detected in a room, **THEN** turn ON the light.
- **IF** temperature is above 28°C, **THEN** turn ON the fan.
- **IF** it is night and motion is detected near the entrance, **THEN** turn ON the security light.
- **IF** smoke is detected, **THEN** activate the alarm.

Advantages

- Simple to design.
- Fast response to real-time events.
- Requires less computational power.
- Useful for straightforward situations.

Limitations

- Cannot remember previous situations.
- Cannot learn user preferences.
- Cannot reason about hidden conditions.

- May make poor decisions when several conditions interact.

Example: If a user normally prefers lights OFF during the afternoon, a simple reflex agent cannot learn this preference unless an additional rule is manually programmed.

2. Model-Based Agent

A **model-based agent** maintains an **internal model or memory of the environment**. It uses both the current input and information from previous situations.

Working in a smart home

Suppose a motion sensor detects movement.

The agent does not simply turn the light ON. It can consider:

- Whether the room is already occupied.
- Whether the light was recently switched OFF.
- The current time.
- Previous motion detected in the room.
- Current temperature and lighting conditions.

For example:

If motion is detected at 11 PM in the living room and the system knows that the user is sleeping, it may avoid turning on bright lights unnecessarily.

Advantages

- Maintains information about previous states.
- Handles partially observable environments.
- Makes better decisions than a simple reflex agent.
- Can represent the current state of the house.

Limitations

- More complex than a reflex agent.
- Requires memory and an accurate model.
- Does not necessarily choose the best action among several possible actions.

3. Goal-Based Agent

A **goal-based agent** chooses actions according to a desired **goal**.

Instead of simply reacting to an event, it asks:

"What action will help me achieve my goal?"

Possible smart-home goals

- Maintain comfortable temperature.
- Reduce electricity consumption.
- Keep the house secure.
- Turn lights off when rooms are unoccupied.
- Prepare the house for the user's arrival.

Example

Suppose the goal is:

"Maintain a comfortable temperature while saving energy."

If the temperature is high, the agent can consider different actions:

- Turn ON the AC.
- Turn ON the fan.
- Open a smart window.
- Reduce the AC temperature slightly.

The agent selects an action that helps achieve the desired goal.

Advantages

- Can plan actions.
- More flexible than rule-based systems.
- Suitable for situations involving multiple steps.
- Can work toward future objectives.

Limitations

- Finding a suitable action may require more computation.
- A goal alone does not indicate which solution is preferable.

For example, both **maximum AC cooling** and **moderate cooling** may achieve the comfort goal, but they have different energy costs.

4. Utility-Based Agent

A **utility-based agent** goes one step further. It evaluates possible actions using a **utility function** and selects an action that provides the highest overall benefit.

In a smart home, utility may combine:

Comfort + Energy Efficiency + Security + User Preference

For example:

Action	Comfort	Energy Use	Security	Overall Utility
--------	---------	------------	----------	-----------------

AC at 18°C High	Very High	Normal	Medium
-----------------	-----------	--------	--------

AC at 24°C High	Moderate	Normal	High
-----------------	----------	--------	------

Fan only	Moderate	Low	Normal	Medium
----------	----------	-----	--------	--------

The agent may select **24°C AC operation** because it provides a better balance between comfort and energy consumption.

Advantages

- Handles conflicting objectives.
- Can make trade-offs.
- Can adapt to user preferences.
- Suitable for complex real-world environments.
- Can optimize comfort, cost, energy, and security simultaneously.

Limitations

- More difficult to design.
- Requires a suitable utility function.
- Requires more computational resources.
- Incorrect utility values can produce undesirable decisions.

Comparison of Intelligent Agents

Agent	Main Decision Method	Memory	Planning	Smart-Home Example
Simple Reflex	Current condition/rule	No	No	Motion → Light ON
Model-Based	Current + previous state	Yes	Limited	Remember room occupancy
Goal-Based	Achieve a goal	Yes	Yes	Maintain comfortable temperature
Utility-Based	Maximize overall benefit	Yes	Yes	Balance comfort and energy

Most Effective Approach for the Smart Home

A **hybrid model-based + goal-based + utility-based agent** would be most effective.

Why?

A smart home is a **dynamic and partially observable environment**. Therefore, one type of agent alone may not be sufficient.

A suitable architecture could work as follows:

Sensors → Model/Memory → Goals → Utility Evaluation → Action

For example:

1. Motion sensor detects a person.
2. The model determines that the person has entered the bedroom.
3. The system checks the time and temperature.
4. The goal is to provide comfort while reducing energy consumption.
5. The utility function compares possible actions.
6. The system turns on a low-intensity light instead of maximum lighting.

Final justification

A **simple reflex agent** is excellent for immediate safety actions such as smoke alarms or emergency lighting. A **model-based agent** provides memory and environmental awareness. A **goal-based agent** provides planning, while a **utility-based agent** allows the system to balance competing requirements.

Therefore, a **hybrid intelligent agent**, particularly one combining **model-based reasoning with goal and utility-based decision-making**, is most suitable for achieving **comfort, energy efficiency, personalization, and security**.

2. ROBOT FINDING AN EXIT IN AN UNKNOWN MAZE

Introduction

Consider a robot placed inside an **unknown maze**. It has no prior map of the maze and must discover the environment while searching for the exit.

This is a search problem because the robot must explore different possible states until it reaches the **goal state**, which is the maze exit.

Two useful uninformed search techniques are:

1. **Uniform Cost Search (UCS)**
2. **Iterative Deepening Search (IDS)**

1. Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the **lowest path cost** first.

It uses a priority queue.

Working

Suppose the robot has several possible paths:

Start

```
|
+---- A ---- Cost 3
|
+---- B ---- Cost 1
|
+---- C ---- Cost 5
```

UCS explores **B first** because its current path cost is the smallest.

The process continues by repeatedly selecting the least-cost path until the exit is reached.

In a maze

If every movement has the same cost, UCS effectively behaves similarly to **Breadth-First Search**.

If different movements have different costs, such as:

- Normal floor = 1
- Mud = 3
- Stairs = 5

then UCS can find the **least-cost path**, not merely the path with the fewest steps.

Advantages of UCS

1. **Complete** when step costs are positive and bounded appropriately.
2. **Optimal** because it finds the least-cost solution.
3. Can handle paths with **different movement costs**.
4. Useful when the robot must consider energy or movement cost.

Disadvantages of UCS

1. Can require a large amount of memory.
2. May explore many unnecessary nodes.

3. Can be slow when the maze is large.
4. Not ideal when all actions have equal cost because simpler methods may be sufficient.

Complexity

For branching factor b , optimal solution cost C^* , and minimum step cost $\epsilon > 0$, UCS has exponential worst-case behaviour, commonly expressed approximately as:

- **Time:** $O(b^{1+\lceil C^*/\epsilon \rceil})$
- **Space:** $O(b^{1+\lceil C^*/\epsilon \rceil})$

The important exam point is that **UCS can consume significant time and memory because it stores many frontier nodes.**

2. Iterative Deepening Search (IDS)

Iterative Deepening Search combines the advantages of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

Instead of searching the entire tree at once, it repeatedly performs depth-limited DFS.

Working

The robot searches with increasing depth limits:

Depth 0 → Start

Depth 1 → Explore one level

Depth 2 → Explore two levels

Depth 3 → Explore three levels

Depth 4 → Continue until exit is found

For example:

Iteration 1: Depth = 0

Iteration 2: Depth = 1

Iteration 3: Depth = 2

Iteration 4: Depth = 3

When the exit is found, the search stops.

Advantages of IDS

1. Requires much less memory than BFS.
2. Complete when the branching factor is finite.
3. Finds the shallowest solution when all step costs are equal.
4. Useful when the depth of the solution is unknown.
5. Particularly suitable for an unknown maze when memory is limited.

Disadvantages of IDS

1. Nodes near the top are expanded repeatedly.
2. This causes additional computation.
3. It does not handle different path costs naturally.
4. If one path is cheap but deep and another is expensive but shallow, IDS may not find the least-cost path.

Complexity

For branching factor **b** and shallowest goal depth **d**:

- **Time:** $O(b^d)$
- **Space:** $O(bd)$

Thus, IDS has **exponential time but relatively low memory requirements**.

UCS vs IDS

Feature	UCS	IDS
Basic idea	Expand lowest-cost path	Increase depth limit gradually
Complete	Yes, under standard positive-cost conditions	Yes, for finite branching
Optimal	Yes, for path cost	Yes for equal step costs
Time	Can be high	$O(b^d)$
Space	High	$O(bd)$
Different path costs	Handles them well	Not suitable

Feature	UCS	IDS
Memory requirement	High	Low
Best use	Cost-sensitive paths	Unknown solution depth with limited memory

Relation to Intelligent Agents

The robot's **agent type affects how it makes decisions.**

Simple Reflex Agent

A simple reflex robot may use rules such as:

IF wall ahead → turn right.

This is fast, but it does not remember where it has already been. Therefore, it may **go around in circles** in a maze.

Model-Based Agent

A model-based robot maintains information about previously visited locations.

For example:

"I have already visited this position, so I should try another unexplored direction."

This makes the robot more suitable for an unknown maze.

Goal-Based Agent

A goal-based robot has a clear objective:

Goal = Reach the exit.

It evaluates possible paths according to whether they lead toward the goal.

Utility-Based Agent

A utility-based robot can consider several factors:

- Distance to exit
- Energy consumption
- Risk
- Path length
- Terrain difficulty

It can select a path that maximizes overall utility.

Best Combination for the Maze

For an **unknown maze with equal movement costs and limited memory**, a **model-based + goal-based agent using IDS** is a strong choice.

Reasoning

Model-based component:

- Records explored locations.
- Maintains information about the discovered maze.
- Avoids repeatedly making the same mistakes.

Goal-based component:

- Defines the exit as the target.
- Guides the search toward finding a solution.

IDS:

- Does not require prior knowledge of the maze depth.
- Uses relatively little memory.
- Can find the shallowest solution when every movement has equal cost.

What if movement costs are different?

If different paths have different costs, such as energy consumption or difficult terrain, then UCS becomes more appropriate because it finds the **least-cost path**.

Final Conclusion

The choice of search algorithm depends on the nature of the maze.

- **IDS** is preferable when movements have equal cost and **memory is limited**.
- **UCS** is preferable when movements have **different costs** and finding the least-cost path is important.
- A **model-based agent** is useful because the maze is initially unknown and the robot needs to maintain information about its environment.
- A **goal-based component** ensures that the robot's decisions are directed toward reaching the exit.

Therefore, a **model-based, goal-oriented robot combined with IDS for equal-cost movement** provides an effective solution for an unknown maze, while **UCS should be selected when path costs differ**.